Parallel Implementation of Particle Storms Using OpenMP+MPI and CUDA

Gianmaria Romano (2105539)

April 6, 2026

1 Introduction

This report analyses parallel implementations of a sequential particle storm simulator that was presented at the EduHPC Peachy Parallel Assignments competition held in 2018 [1], focusing on two main strategies: a hybrid OpenMP+MPI model and a CUDA implementation.

1.1 Sequential code analysis

The baseline sequential code found in the `energy_storms_core.c` file structures the storm simulation into three main steps:

1. The bombardment phase, which is implemented as a triple nested loop that iterates through N waves, P particles per wave and K layer cells, for an approximate complexity of $O(N \times P \times K)$.
2. The relaxation phase, which is implemented using two distinct loops that iterate through the layer cells, for a complexity of $O(K)$ per wave.
3. The maximum detection phase, which is implemented as a loop that iterates through the layer cells to find the global maximum, for a complexity of $O(K)$ per wave.

While simple and functionally correct, the sequential algorithm is very computationally expensive.

In fact, each wave requires $O(P \times K)$ calls to the `update` function, which, however, makes use of compute-heavy operations, such as `sqrtf`, that might be redundant in some steps.

In addition, the relaxation phase introduces another computational bottleneck by requiring a $O(K)$ copy of the values to an ancillary array to avoid race conditions.

Furthermore, the maximum detection phase hinders CPU efficiency by overusing selective statements to check for local maxima.

2 OpenMP + MPI Implementation

The hybrid implementation, which is found in the `energy_storms_mpi_omp_core.c` file, uses the Message Passing Interface library[2] for domain decomposition and communication among working processes and the OpenMP API[3] to parallelize compute-heavy tasks by distributing the workload among threads.

2.1 Initialization and setup

The input array is partitioned into local chunks that are distributed among processes, each working on a set range of cells, handling eventual workload unevenness using a remainder calculation.

```
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
local_array_size = layer_size / comm_sz;
remainder = layer_size % comm_sz;
if (my_rank < remainder) {
    // The first remainder processes handle an extra cell.
    local_array_size = local_array_size + 1;
    local_start = my_rank * local_array_size;
} else {
    local_start = (my_rank * local_array_size) + remainder;
}
local_end = local_start + local_array_size;
```

In addition, to eliminate the bottleneck caused by copying values into an ancillary array for the relaxation phase, this program uses two buffers to coordinate a pointer swap and, to encourage vectorization, allocates memory using `posix_memalign`, which ensures cache line alignment to reduce the number of memory fetches and improve SIMD efficiency.

```
float *layer, *layer_copy;
size_t alignment = 64;
size_t size = (layer_size + 2) * sizeof(float);
if (posix_memalign((void**)&layer, alignment, size) != 0 ||
      posix_memalign((void**)&layer_copy, alignment, size) !=
      0) {
    fprintf(stderr, "Error: Allicating the layer memory\n");
    exit(EXIT_FAILURE);
}
// Initialize to 0 as posix_memalign does not clean memory.
memset(layer, 0, size);
memset(layer_copy, 0, size);
// Initialize the pointers on the actual starting cells.
float *current_ptr = &layer[1];
float *copy_ptr = &layer_copy[1];
```

2.2 The parallel implementation

The hybrid model focuses on maximizing the throughput of the `core` function by overlapping, where possible, communication and computation and by introducing inlining and SIMD vectorization to reduce computational overhead.

2.2.1 The bombardment phase

The bombardment phase is now structured as a parallel loop that uses guided scheduling in order to dis-

tribute uneven workload in a more balanced way.

To make better use of cache locality, the inner loops are flipped, which means that the value of each layer cell is updated with respect to all particles of the current storm.

Additionally, to encourage vectorization and avoid redundant calls, the main commands of the `update` function, except for the threshold check as it turned out not to influence the final results, are inlined into the innermost loop and, to avoid issues with memory alignment, the `safelen` clause[4] is introduced to limit how many calculations the compiler can perform at once.

```
#pragma omp parallel for schedule(guided) private(j)
for (k = local_start; k < local_end; k++) {
    #pragma omp simd safelen(8)
    for (j = 0; j < storms[i].size; j++) {
        float energy = (float)storms[i].posval[j*2+1] * 1000;
        int position = storms[i].posval[j*2];
        int distance = abs(position - k) + 1;
        float atenuacion = sqrtf((float)distance);
        float energy_k = energy / layer_size / atenuacion;
        current_ptr[k] += energy_k;
    }
}
```

2.2.2 The relaxation phase

Since the relaxation phase requires "fixing" values based on a cell's immediate neighbours, the processes will need to exchange boundary values to ensure that computations are done correctly.

To reduce the communication bottleneck, the program uses non-blocking communication to overlap boundary exchanges with internal relaxation calculations, which do not involve the boundary cells, on the destination buffer.

```
MPI_Request reqs_copy[4];
for (int r = 0; r < 4; r++) {
    // Note that not all processes do 4 exchanges.
    reqs_copy[r] = MPI_REQUEST_NULL;
}
int counter = 0;
if (my_rank > 0) {
    // Left exchange.
    MPI_Irecv(&current_ptr[local_start - 1], 1, MPI_FLOAT,
          my_rank - 1, 0, MPI_COMM_WORLD, &reqs_copy[counter
          ++]);
    MPI_Isend(&current_ptr[local_start], 1, MPI_FLOAT, my_rank
           - 1, 0, MPI_COMM_WORLD, &reqs_copy[counter++]);
}
if (my_rank < comm_sz - 1) {
    // Right exchange.
    MPI_Irecv(&current_ptr[local_end], 1, MPI_FLOAT, my_rank +
           1, 0, MPI_COMM_WORLD, &reqs_copy[counter++]);
    MPI_Isend(&current_ptr[local_end - 1], 1, MPI_FLOAT,
          my_rank + 1, 0, MPI_COMM_WORLD, &reqs_copy[counter
          ++]);
}
// Non-boundary relaxation before synchronization.
#pragma omp parallel for schedule(static, 1024)
for (k = aux_start + 1; k < aux_end; k++) {
    copy_ptr[k] = (current_ptr[k - 1] + current_ptr[k] +
          current_ptr[k + 1]) / 3;
}
MPI_Waitall(counter, reqs_copy, MPI_STATUSES_IGNORE);
```

Following the boundary cell exchange, each process is able to complete the relaxation phase by working on the boundaries of its chunk.

```
copy_ptr[aux_start] = (current_ptr[aux_start - 1] +
      current_ptr[aux_start] + current_ptr[aux_start + 1]) /
      3;
if (aux_end - 1 > aux_start) {
    // Only if feasible.
    copy_ptr[aux_end - 1] = (current_ptr[aux_end - 2] +
          current_ptr[aux_end - 1] + current_ptr[aux_end]) /
          3;
}
if (my_rank == 0) {
    // Copy cell 0.
    copy_ptr[0] = current_ptr[0];
}
if (my_rank == comm_sz - 1) {
    // Copy cell layer_size - 1.
    copy_ptr[layer_size - 1] = current_ptr[layer_size - 1];
}
```

At the end of the relaxation phase, a pointer swap allows the program to quickly recover the post-relaxation values, eliminating the need of a redundant copy loop, and a second boundary exchange is performed before the maximum detection phase.

```
float *to_swap = current_ptr;
current_ptr = copy_ptr;
copy_ptr = to_swap;
```

2.2.3 The maximum detection phase

The maximum detection phase is implemented as two-step reduction that uses a dedicated structure to store information about maximum points.

```
struct {
    float max_val;
    int max_pos;
} local_max, global_max;
```

First, each OpenMP thread finds a thread-local maximum and, using a critical section, each MPI process is able to use these values to find the local maximum within its chunk.

```
#pragma omp parallel
{
    float t_max_val = -1.0f;
    int t_max_pos = -1;
    #pragma omp for schedule(static)
    for (k = max_start; k < max_end; k++) {
        if (current_ptr[k] > current_ptr[k - 1] && current_ptr
              [k] > current_ptr[k + 1] && current_ptr[k] >
              t_max_val) {
            t_max_val = current_ptr[k];
            t_max_pos = k;
        }
    }
    #pragma omp critical
    {
        if (t_max_val > local_max.max_val) {
            local_max.max_val = t_max_val;
            local_max.max_pos = t_max_pos;
        }
    }
}
```

Then, the global maximum is found by means of the `MPI_Allreduce()` call, which allows processes to exchange both the value and the position of their local maxima by means of the `MPI_MAXLOC` operator.

```
MPI_Allreduce(&local_max, &global_max, 1, MPI_FLOAT_INT,
      MPI_MAXLOC, MPI_COMM_WORLD);
maximum[i] = global_max.max_val;
positions[i] = global_max.max_pos;
```

2.3 Previous implementations

Most experiments on this program dealt with improving the relaxation phase and addressing potential bottlenecks caused by the critical section in the maximum detection phase.

Concerning the former, boundary exchanges were initially performed using the `MPI_Sendrecv()` call, which, however, caused a bottleneck due to its blocking nature, leading to opt for non-blocking communication using `MPI_Isend()` and `MPI_Irecv()` instead.

This change alone, however, was still not sufficient due to performing all relaxation calculations after the exchange and keeping the array copy loop, leading to the implementation of the choices mentioned in the previous sections.

Regarding the critical section, an alternative idea consisted of trying a custom OpenMP reduction.

```
typedef struct {
    float max_val;
    int max_pos;} MaxLoc;
#pragma omp declare reduction(maxloc : MaxLoc : omp_out = (
      omp_in.max_val > omp_out.max_val) ? omp_in : (omp_in.
      max_val == omp_out.max_val \&& omp_in.max_pos < omp_out.
      max_pos) ? omp_in : omp_out) initializer(omp_priv =
      {-1.0f, -1})
```

While functionally correct, this strategy was not implemented in the final version of the program as it did not appear to improve its overall performance.

2.4 OpenMP + MPI performance analysis

Performance analysis for the hybrid model focused both on inter-node scaling, which was studied by taking measurements using one and four processes, and intra-node scaling, which was studied by taking measurements across different number of threads.

In particular, the program was compiled using some flags, which are presented in better detail in Appendix B.1, to achieve maximum optimization.

2.4.1 Speedup analysis

Equation 1 measures the speedup of a parallel application as the performance improvement that is obtained by going from the sequential baseline to a parallel algorithm that is executed using p processes and t threads.

$$Speedup(p,t) = \frac{T_{seq}}{T_{par}(p,t)} \tag{1}$$

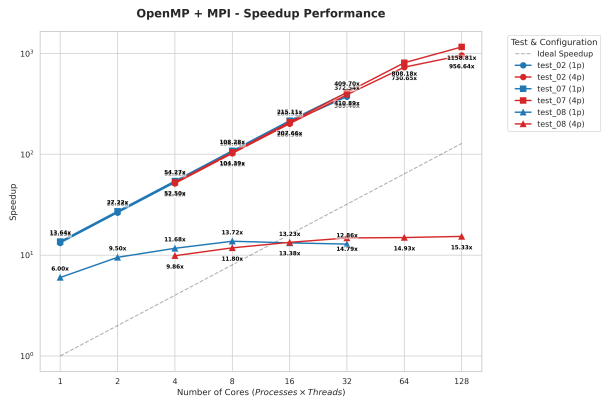

Figure 1: Speedup across cores for `test_02`, `test_07` and `test_08`.

The results reported in Figure 1 suggest that using a single process tends to be more convenient when fewer threads are used, while using four processes yields better results when more threads are introduced, although the overall performance pattern depends on the workload of each test.

In fact, "heavier" tests, such as `test_02` and `test_07`, allow the program to achieve massive, super-linear speedups, peaking at **956.64x** and **1158.81x** respectively for the 128-core setting. Such results can be attributed to "cache-effect scalability" as performing domain decomposition among more processes allows each process to fit its chunk in the L3 cache, significantly reducing memory access latencies in favour of actual computations.

By contrast, `test_08` causes the program to achieve much worse results, stalling at a speedup of **15.33x** for the 128-core setting. This result can be explained by the sparse structure of `test_08`: despite working on a massive array, this test has the program process a very small number of particles, causing parallel overhead to shadow performance gains.

2.4.2 Efficiency analysis

Equation 2 measures the efficiency of a parallel application as how well the parallel program uses the available resources.

$$Efficiency(p,t) = \frac{Speedup(p,t)}{pt} = \frac{T_{seq}}{pt \cdot T_{par}(p,t)} \tag{2}$$

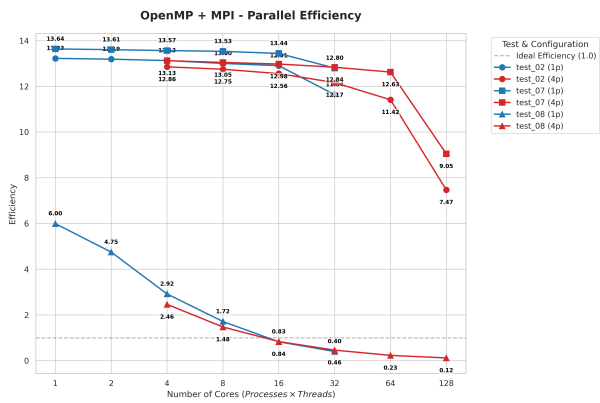

Figure 2: Efficiency across cores for `test_02`, `test_07` and `test_08`.

Figure 2 appears to confirm the results discussed in Section 2.4.1.

In fact, `test_02` and `test_07`, which benefitted from super-linear speedups, allow the program to maintain consistently high efficiency, although this metric drops sharply when many cores are employed, likely due to the increasing parallel overhead for these settings.

On the other hand, the results reported for `test_08` reveal that using many resources on sparser tests is counterproductive as the program efficiency

quickly drops below the ideal efficiency threshold, plummeting at **0.12** when 128 cores are used.

2.4.3 Scalability analysis

Equation 3 measures the scalability of a parallel application as the performance improvement that is obtained as the parallel program uses more cores.

$$Scalability(p,t) = \frac{T_{par}(1,1)}{T_{par}(p,t)} \tag{3}$$

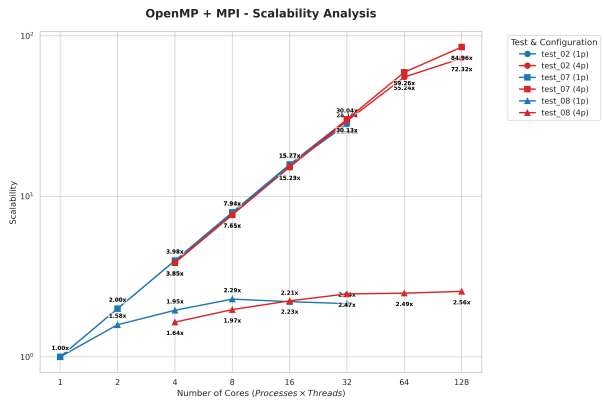

Figure 3: Scalability across cores for `test_02`, `test_07` and `test_08`.

Figure 3 suggests that the program is able to scale strongly as more cores are used, although, again, the overall trend depends on the nature of each test.

In fact, `test_02` and `test_07` allow the program to initially achieve quasi-linear scaling, reaching relative speedups of around **30x** for 32-core settings, although scaling slows down when further increasing cores, stopping at **72.32x** and **84.96x** respectively for the 128-core setting, meaning that the program suffers from oversubscription as the parallel overhead starts to outweigh computational improvements.

By contrast, the sparse structure of `test_08` causes the program to struggle with scalability, which never exceeds the relative speedup of **2.56x** achieved in the 128-core setting, as its small workload hinders the program from achieving further improvements.

2.4.4 Profiling using Performance Counters for Linux

The hybrid implementation was profiled using Linux's `perf`[5] command, which uses dedicated registers to analyse hardware performance metrics, such as the number of instructions executed in a single cycle or the number of cache misses, that can help interpret the program's scaling trends.

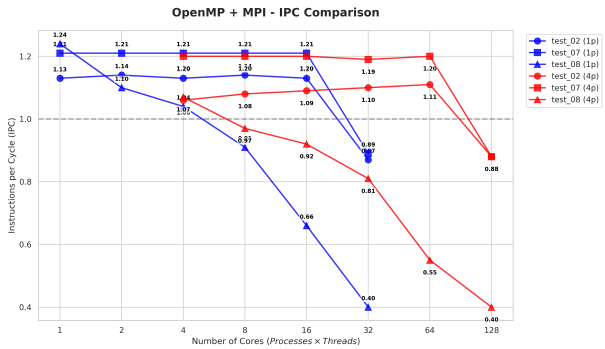

Figure 4: IPC measures across cores for `test_02`, `test_07` and `test_08`.

For instance, Figure 4 suggests that using a single process allows the program to increase the instructions it can execute in a single cycle as there is no communication overhead, although both settings ultimately fall down the baseline of 1 instruction per cycle when many threads are used, likely due to the overhead introduced by the critical section in the maximum detection phase.

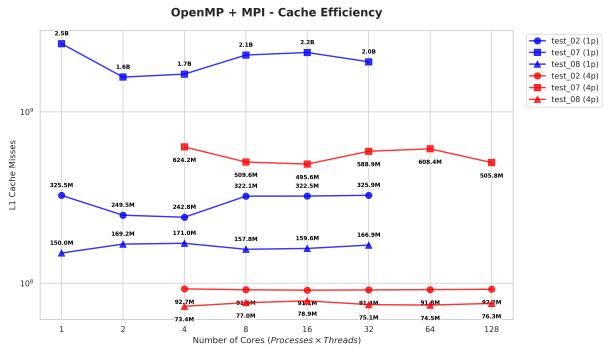

Figure 5: Cache misses across cores for `test_02`, `test_07` and `test_08`.

By contrast, Figure 5 reveals that using four processes helps the program significantly reduce cache misses as domain decomposition into local arrays allows each process to fit its chunk into the cache, reducing latencies caused by cache misses and improving scaling.

3 CUDA Implementation

The CUDA[6] implementation, which is found in the `energy_storms_cuda_core.cu` file, mostly performs computations on the GPU by implementing each phase of the simulation as a dedicated kernel and, to reduce the overhead caused by data transfers, introduces shared memory usage for the relaxation and maximum detection phases.

3.1 The bombardment kernel

The bombardment phase is structured as a kernel that, by calling the `update` function as a device ker-

nel, processes the impact that each particle in the current storm has on the same layer cell `k`.

When this kernel has processed the entire array, the program later copies the obtained results on the ancillary array using a `cudaMemcpy()` call.

```
__global__ void update_storm(float *layer, int layer_size,
      int *posval, int particles) {
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    if (k < layer_size) {
        // Keep working on cell k by doing each single update.
        for (int p = 0; p < particles; p++) {
            // Get particle information.
            int position = posval[p * 2];
            float energy = (float)posval[p * 2 + 1] * 1000;
            update(layer, layer_size, k, position, energy);
        }
    }
}
```

In order to implement this logic, the program makes use of an array that stores all the information of the current storms, allocating the maximum amount of memory that is needed by a single storm in order to avoid memory management issues.

```
int max_particles = 0;
for (i = 0; i < num_storms; i++) {
    if (storms[i].size > max_particles) {
        max_particles = storms[i].size;
    }
}
int *device-storm_particles;
// Allocate space for position and energy of each cell.
cudaMalloc(&device_storm_particles, sizeof(int) *
      max_particles * 2);
```

Overall, this implementation allows the program to achieve better performance as it reduces the number of kernel calls to just one per wave, significantly reducing the resulting overhead.

3.1.1 Previous implementations

The initial implementation of the bombardment phase consisted of a kernel that updated the energy of a layer cell with respect to the impact of a single particle.

While much simpler to implement, this strategy turned out to be less computationally advantageous due to the kernel launch overhead caused by the need to call the function $O(P \times K)$ times.

3.2 The relaxation kernel

Since reading each layer cell's immediate neighbours from the global memory tends to be computationally expensive, the relaxation phase makes use of the shared memory by caching the cells for a block and two extra cells for the neighbours of the block-local boundaries.

```
// Allocating (BLOCK_SIZE + 2) comes from handling the
      boundaries of each block.
size_t shared_memory_size_relaxation = (BLOCK_SIZE + 2) *
      sizeof(float);
relaxation<<<n_blocks, BLOCK_SIZE,
      shared_memory_size_relaxation>>>(device_layer,
      device_layer_copy, layer_size);
```

In this way, each thread can load its data point and its direct neighbours, eventually using a zero-padding placeholder if such neighbours are actually out-of-bounds positions, and, after using a synchronization barrier to ensure data consistency, the kernel can exploit the shared memory to perform the relaxation calculations.

```
__global__ void relaxation(float *layer, float *layer_copy,
      int layer_size) {
    extern __shared__ float shared_data[];
    int thread_id = threadIdx.x;
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    if (k < layer_size) {
        // Take the value found in cell k.
        shared_data[thread_id + 1] = layer_copy[k];
        if (thread_id == 0) {
            /* First thread in the block.
            Take the left neighbour. */
            if (k > 0) {
                // Left neighbour.
                shared_data[0] = layer_copy[k - 1];
            } else {
                // Placeholder (no left neighbour).
                shared_data[0] = 0.0f;
            }
        }
        if (thread_id == blockDim.x - 1 || k == layer_size -
              1) {
            /* Last thread in the block.
            Take the right neighbour from the next block. */
            if (k < layer_size - 1) {
                // Right neighbour.
                shared_data[thread_id + 2] = layer_copy[k + 1];
            } else {
                // Placeholder (no right neighbour).
                shared_data[thread_id + 2] = 0.0f;
            }
        }
    } else {
        // Placeholder for invalid positions.
        shared_data[thread_id + 1] = 0.0f;
    }
    __syncthreads(); // Thread synchronization.
    // Relaxation step.
    if (k > 0 && k < layer_size - 1) {
        // Skip updating the first and last cells.
        layer[k] = (shared_data[thread_id] + shared_data[
              thread_id + 1] + shared_data[thread_id + 2]) / 3;
    }
}
```

3.3 The maximum detection phase

Similarly to the hybrid implementation discussed in Section 2.2.3, the maximum detection phase is structured in two main steps.

In fact, the `local_max_reduction` kernel implements a tree reduction with shared memory that allows the threads in a block to find the local maximum within its local chunk, storing it in a dedicated array.

Then, the `global_max_reduction` kernel implements another tree reduction with shared memory to identify the global maximum, whose information are ultimately transferred on the CPU.

```
// Allocating (sizeof(float) + sizeof(int)) comes from
      handling both the values and their positions.
size_t shared_memory_size_max_reduction = BLOCK_SIZE * (
      sizeof(float) + sizeof(int));
local_max_reduction<<<n_blocks, BLOCK_SIZE,
      shared_memory_size_max_reduction>>>(device_layer,
      device_max_val, device_max_pos, layer_size);
global_max_reduction<<<1, BLOCK_SIZE,
      shared_memory_size_max_reduction>>>(device_layer,
      device_max_val, device_max_pos, layer_size);
cudaMemcpy(&maximum[i], device_global_max_val, sizeof(float),
       cudaMemcpyDeviceToHost);
cudaMemcpy(&positions[i], device_global_max_pos, sizeof(int),
       cudaMemcpyDeviceToHost);
```

3.3.1 Local maximum kernel

After partitioning the allocated memory to keep track of values and positions, each thread uses the shared memory to find a thread-local maximum and, after a synchronization barrier to ensure data consistency, the `local_max_reduction` kernel performs a tree reduction to aggregate such maxima into a block-level local maximum whose information are stored on a dedicated GPU array at the end of the reduction.

```
__global__ void local_max_reduction(float *layer, float *max_val, int *max_pos, int layer_size) {
    // Partition the allocated memory.
    extern __shared__ float shared_max[];
    int *shared_pos = (int *)&shared_max[blockDim.x];
    int thread_id = threadIdx.x;
    int k = blockIdx.x * blockDim.x + threadIdx.x;
    shared_max[thread_id] = -1.0f;
    shared_pos[thread_id] = -1;
    if (k > 0 && k < layer_size - 1) {
        // Check it only if it is a local maximum.
        if (layer[k] > layer[k - 1] && layer[k] > layer[k + 1]) {
            shared_max[thread_id] = layer[k];
            shared_pos[thread_id] = k;
        }
    }
    __syncthreads();
    // Tree reduction for block-wide local maximum.
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (thread_id < s) {
            if (shared_max[thread_id + s] > shared_max[thread_id]) {
                shared_max[thread_id] = shared_max[thread_id + s];
                shared_pos[thread_id] = shared_pos[thread_id + s];
            }
        }
        __syncthreads();
    }
    // Thread 0 writes the local maximum and its position.
    if (thread_id == 0) {
        max_val[blockIdx.x] = shared_max[0];
        max_pos[blockIdx.x] = shared_pos[0];
    }
}
```

3.3.2 Global maximum kernel

After the first tree reduction has been completed, the `global_max_reduction` kernel carries out a second tree reduction that, following the same logic as the `local_max_reduction` kernel, aggregates block-wide local maxima into a global maximum that is stored in a dedicated GPU array at the end of the reduction.

At the end, the program ultimately copies the information of the global maximum on the host memory using the `maximum` and `positions` arrays.

```
cudaMemcpy(&maximum[i], device_global_max_val, sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(&positions[i], device_global_max_pos, sizeof(int), cudaMemcpyDeviceToHost);
```

3.4 CUDA performance analysis

For a more comprehensive study, performance analysis for the CUDA implementation was conducted by passing different block sizes in the `core` function in order to understand which configuration can be the most beneficial.

3.4.1 Speedup across block sizes

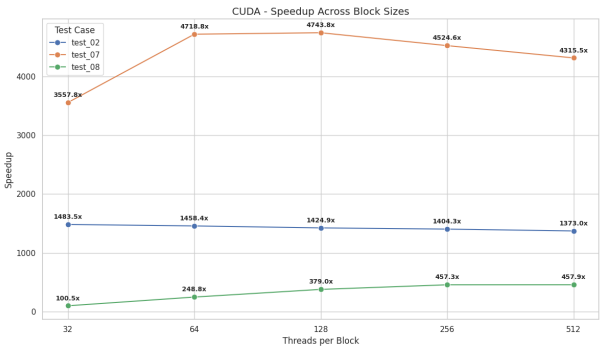

Figure 6: Speedup across block sizes for `test_02`, `test_07` and `test_08`.

Figure 6 shows that the program is able to achieve very high speedups, although the overall results and trends for each test vary depending on how well the GPU is able to hide eventual latencies.

For instance, the large workload provided by `test_07` allows the program to achieve massive speedups, peaking at **4743.75x** when blocks of 128 threads are used, as the GPU is able to schedule and exploit active warps even while other warps are waiting for higher-latency tasks, such as global memory accesses.

On the other hand, the speedup achieved for `test_02` appears to decrease as the block size increases, starting at **1483.51x** with blocks of 32 threads and falling at **1372.95x** with blocks of 512 threads, because, since this test has a smaller workload compared to `test_07`, using larger blocks may hinder GPU concurrency due to memory contention between warps.

Lastly, the sparse structure of `test_08` causes the program to be more memory-bound as, due to the much smaller workload of this test, its performance appears to be limited more by memory bandwidth rather than by the GPU's computational power, although the positive correlation between block size and speedup, which starts at **100.5x** with blocks of 32 threads and reaches **457.87x** with blocks of 512 threads, might suggest that creating more warps can help the program hide latencies for sparser tests.

3.4.2 Profiling using Nsight Systems

The CUDA implementation was profiled using NVIDIA Nsight Systems[7], which comes in handy to analyse bottlenecks in CPU-GPU interactions.

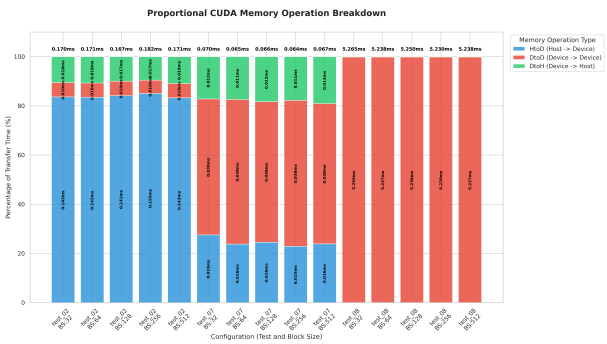

Figure 7: Analysis of `cudaMemcpy` operations across block sizes for `test_02`, `test_07` and `test_08`.

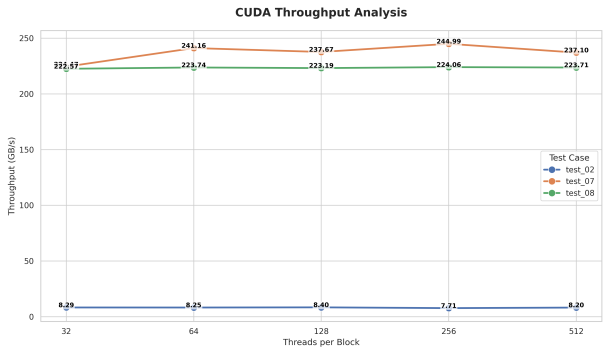

Figure 8: Throughput across block sizes for `test_02`, `test_07` and `test_08`.

Figures 7 and 8 describe how, for the analysed tests, the program behaves in regards to memory operations and throughput.

In fact, memory operations for `test_07` mostly consist of device-to-device data transfers, which outweigh the overhead introduced by host-device `Memcpy` calls and allow the GPU to remain more active, as reflected by the high throughput achieved by the program for this test.

By contrast, the execution of `test_08`, despite the high throughput and the predominance of device-to-device data transfers, is more-memory bound because, due to the small workload to process, the program actually spends proportionally more time on memory operations, which helps explain why the speedup achieved by the program for this test grows less steadily compared to other tests.

Similarly, the program's low throughput for `test_02` might suggest that its execution is also memory-bound as the program mostly moves the data from the host memory to the GPU and the negative correlation between block size and throughput appears to suggest that using more warp hinders GPU utilization for this test.

Attempts to improve memory transfers included page-locked memory management[8] and asynchronous memory management with streams[9], although neither of these choices appeared to provide significant advantages.

4 Overall Performance

Each performance analysis discussed in Sections 2.4 and 3.4 was conducted by taking five measurements per configuration in order to address execution time variability and work with more robust results.

In fact, together with the execution times, the average, minimum and maximum times, along with their standard deviation, have been recorded in order to assess both the central behaviour and dispersion of their distributions.

4.1 Comparisons across test files

As detailed in Appendix A, the project provides several test cases serving different purposes, ranging from debugging to performance analysis.

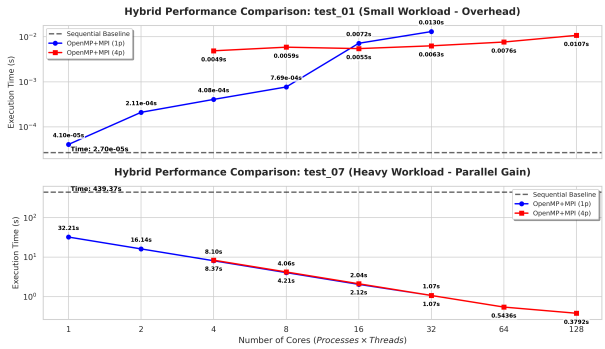

Figure 9: OpenMP + MPI performance comparison between `test_01` and `test_07`.

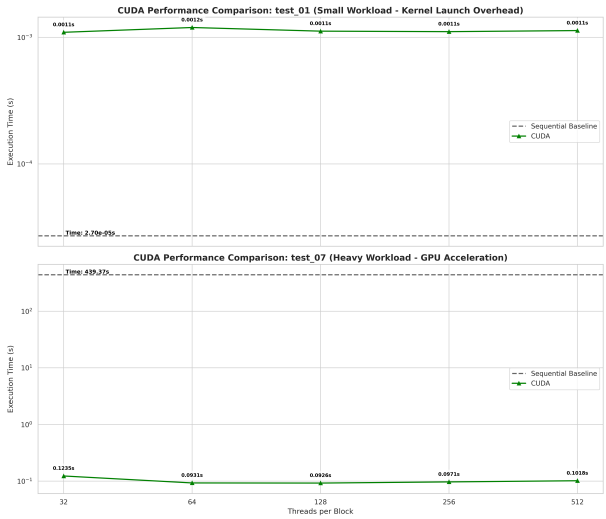

Figure 10: CUDA performance comparison between `test_01` and `test_07`.

For instance, Figures 9 and 10 compare the performance of the parallel implementations between `test_01`, which is used for debugging, and `test_07`, which provides a significant amount of workload, with respect to their sequential baselines.

In particular, `test_01`, which provides a very small workload, suffers from parallel overhead, which is

caused by initializing the MPI environment or launching CUDA kernels, that overshadows potential performance advantages, causing the parallel programs to have worse execution times compared to the sequential baseline.

By contrast, `test_07`, which provides the most workload among the available tests, allows the parallel implementations to do significantly better than the sequential baseline by distributing work among processes and threads, therefore hiding overhead-introduced latencies.

4.2 Comparisons across implementations

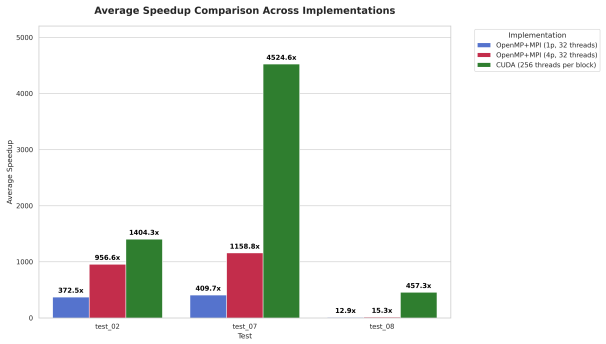

Figure 11: Speedup comparisons between some configurations for `test_02`, `test_07` and `test_08`.

Figure 11 shows that, on average, the CUDA implementation achieves significantly better results compared to the hybrid OpenMP + MPI implementation.

In fact, while the hybrid model is able to reduce execution times by exploiting SIMD vectorization and distributed memory usage, the CUDA implementation is able to better overcome computational bottlenecks by mapping threads to layer cells during the bombardment phase, which allows the program to exploit coalesced memory accesses, and introducing shared memory usage to improve the relaxation and maximum detection phases.

Regarding the hybrid implementation, the analysis conducted in Section 2.4 shows that, for equal core combinations, the configurations using four processes cause the program to suffer more from communication overhead, as reflected by metrics like efficiency or instructions per cycle, despite achieving similar results to the configurations using a single process.

5 Conclusions

Analysing the parallel implementations of the particle storm simulator confirms that this problem is highly suitable for (intensive) GPU computations due to the logic behind the layer cells updates.

In fact, the results describes in the previous sections demonstrate that, while the hybrid model is able to achieve strong scalability when working on larger tests, the CUDA implementation tends to provide the overall best results as the GPU can better hide its memory latencies by exploiting both thread parallelism and low-latency shared memory.

Overall, future optimizations should focus on improving hardware utilization to overcome parallel overhead for sparse tests, such as `test_08`, in order to achieve better and more consistent results.

Concerning the parallel programs, future work can regard finding a suitable trade-off between instructions per cycle and cache utilization for the hybrid model and improving memory management and data transfers for the CUDA implementation.

References

[1] A. González-Escribano, "Storms of high-energy particles: A peachy parallel assignment," in *Proceedings of the Workshop on Education for High-Performance Computing (EduHPC '18)*, IEEE/ACM, 2018.

[2] Message Passing Interface Forum, "MPI: A message-passing interface standard." `https://www.mpi-forum.org/docs/mpi-1.1/mpi-11-html/mpi-report.html`, 1994.

[3] OpenMP Architecture Review Board, "OpenMP application programming interface version 5.0." `https://www.openmp.org/specifications/`, 2018.

[4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Version 5.0*, November 2018. Available at `https://www.openmp.org/spec-html/5.0/openmp.html`.

[5] The Perf Community, "perf: Linux profiling with performance counters." `https://perfwiki.github.io/main/`, 2024. Accessed: 2024-05-21.

[6] NVIDIA Corporation, "NVIDIA CUDA C Programming Guide." `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2025.

[7] NVIDIA Corporation, *Nsight Systems User Guide*, 2026. v2026.1, Accessed: 2026-03-04.

[8] NVIDIA Corporation, *CUDA C++ Programming Guide*. NVIDIA Corporation, 2026. Section 2.4.3: Page-Locked Host Memory.

[9] NVIDIA Corporation, "Streams and concurrency: CUDA training webinar." NVIDIA Developer Training, 2024. Accessed: 2024-05-20.

A Details about datasets

Several datasets with specific characteristics and purposes have been used for testing the codes.

Test	Waves	Total Particles	Layer Size
`test_01`	4	28	35
`test_02`	6	120000	20000 or 30000
`test_03`	1	4	20
`test_04`	1	4	20
`test_05`	1	4	20
`test_06`	1	4	20
`test_07`	4	20000	1000000
`test_08`	3	3	100000000
`test_09`	1	3	16 or 17

Table 1: Dataset size for each test.

In particular, `test_02`, `test_07` and `test_08` provide enough workload to test for performance improvements and efficiency of the parallel implementations, whereas the remaining tests are mainly used for debug-related tests.

B Details about compilation

In order to give a further performance boost, the parallel implementations use some compilation flags that allow the system to fully exploit its capabilities.

B.1 Compilation flags for the Hybrid Implementation

The hybrid OpenMP+MPI implementation was compiled using the following flags:

- `-O3`: this flag activates general optimization through vectorization, inlining and loop unrolling.
- `-march=native`: this flag compiles the code in such a way that maximizes CPU activity.
- `-fno-math-errno`: this flag disables the creation and writing of the `errno` variable in order to safely parallelize mathematical operations.
- `-fno-trapping-math`: this flag lets the compiler assume that mathematical operations never cause exceptions, such as zero division or overflow, allowing to make execution faster.
- `-fstrict-aliasing`: this flag improves memory management for the pointer swap technique by making sure that the pointers always indicate non-overlapping memory regions.

Additional flags, such as `-funroll-loops` or `-ffast-math`, have been tried as well, although they either hindered performance or caused assertion errors during computations.

B.2 Compilation flags for the CUDA Implementation

While the `-O3` flag has been used to compile the CUDA implementation, it did not seem to bring any advantage in terms of performance.